

Reading Motor Encoder Data

Experiment Objective

This section will guide you through reading the pulse data of the motor encoders on the MicroROS-V2 control board. Through this tutorial, you will learn how to use the ESP32-S3's PCNT (Pulse Count) peripheral to perform quadrature decoding of the encoder A/B phase signals, obtain the motor's rotation direction and position information in real time, laying the foundation for subsequent speed closed-loop control (PID).

Experiment Principle

1. Keywords

Keyword	Description
Encoder	A sensor that converts the motor's rotation position and speed into electrical signals, usually outputting two quadrature pulse signals, A and B
Quadrature Decoding	Determines both the rotation direction and the pulse count by analyzing the two pulse signals A/B that differ in phase by 90°
PCNT (Pulse Count)	The ESP32-S3's pulse counter peripheral, which supports quadrature decoding mode and glitch filtering, with configurable upper/lower limits and automatic accumulation
Accum Count	When the PCNT count exceeds the upper limit or falls below the lower limit, it automatically resets and continues counting, enabling infinite-range pulse accumulation
Glitch Filter	Filters out pulse interference signals narrower than the set value, improving the stability and reliability of encoder readings
Watch Point	The PCNT triggers an event or resets when the count reaches a preset upper/lower limit, used to manage the counting range

2. Technical Architecture

The program performs quadrature decoding of the two motor encoder signals using the ESP32-S3's PCNT peripheral. The M1 encoder is connected to GPIO5 (phase A) and GPIO4 (phase B); the M2 encoder is connected to GPIO7 (phase A) and GPIO6 (phase B). The PCNT is configured in accumulate count mode with upper/lower limits of ± 1000 , and the glitch filter is enabled (filtering interference $< 1000\text{ns}$). The forward/reverse direction is automatically identified by judging the phase relationship between A/B. The main loop reads the accumulated pulse count of the encoder every 200 ms and prints it.

Main component files:

- `main/main.c` — main program entry, reads and prints encoder data every 200 ms
- `components/encoder/encoder.c` / `encoder.h` — encoder driver implementation and interface declarations

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable	✓ Required (for power supply and serial monitor)
Battery (12V)	✓ Required (to power the motors)
MicroROS-V2 chassis motor with encoders	✓ Required (encoder signals already connected to the control board)

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Battery connected to the control board power interface
4. The chassis motor encoder interfaces are connected to the control board (connected by default at the factory)

Encoder pin definitions

The correspondence between the encoder interfaces on the MicroROS-V2 control board and the GPIO is as follows:

Encoder interface	Pin A	Pin B
M1 encoder	GPIO5 (H1A)	GPIO4 (H1B)
M2 encoder	GPIO7 (H2A)	GPIO6 (H2B)

3. Compile and Flash

```
cd ~/esp/base_samples/encoder
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

After the program starts, it automatically initializes the PCNT counter modules of the two encoders and reads the accumulated pulse counts of the M1 and M2 encoders every 200 ms, outputting them to the serial monitor via logs.

To observe changes in the encoder data, you can run the Motor Driving example (Section 13.9) at the same time to make the motors rotate, and then view the encoder readings.

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After correctly flashing and running the program, the serial monitor outputs the following logs:

```
hello yahboom
I (xxx) MAIN: Nice to meet you!
I (xxx) Encoder: Init pcnt driver to decode
I (xxx) MAIN: Encoder:0, 0
I (xxx) MAIN: Encoder:0, 0
...
```

When the motors are stationary, the encoder values are 0.

If you rotate the motor wheels manually or run the Motor Driving example at the same time, the encoder values will change with the rotation:

```
I (xxx) MAIN: Encoder:156, -142
I (xxx) MAIN: Encoder:312, -285
I (xxx) MAIN: Encoder:468, -428
```

- Forward rotation increases the count (positive values)
- Reverse rotation decreases the count (negative values)
- The signs of the M1 and M2 values are opposite because the two motors are mounted in a mirror configuration (consistent with the `PWM_MOTOR_LOGIC_SIGN` logic direction in the motor driver)

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/encoder/main/main.c`
- Encoder driver: `~/esp/base_samples/encoder/components/encoder/encoder.h`, `encoder.c`

1. Project Structure

```
encoder/
├── CMakeLists.txt           # Top-level CMake project file
├── main/
│   ├── CMakeLists.txt       # main component build configuration
│   └── main.c               # Main program entry
├── components/
│   └── encoder/
│       ├── CMakeLists.txt    # Component configuration (depends on driver)
│       ├── encoder.c         # Encoder driver implementation
│       └── encoder.h         # Encoder driver interface declarations
```

2. Main Program Logic

The execution flow of `app_main` in `main.c`:

1. Print the startup information
2. Call `Encoder_Init()` to initialize the two encoders
3. Delay 1 second to allow the encoders to stabilize
4. Enter the main loop, calling `Encoder_Get_Count_M1()` and `Encoder_Get_Count_M2()` every 200 ms to get the encoder pulse counts and print them

3. Key Components

Encoder Driver (`encoder.h` / `encoder.c`)

Pin and PCNT parameter definitions:

Parameter definition file: `~/esp/base_samples/encoder/components/encoder/encoder.h`

```
#define ENCODER_GPIO_H1A      5      // M1 encoder phase A
#define ENCODER_GPIO_H1B      4      // M1 encoder phase B

#define ENCODER_GPIO_H2A      7      // M2 encoder phase A
#define ENCODER_GPIO_H2B      6      // M2 encoder phase B

#define ENCODER_PCNT_HIGH_LIMIT 1000  // PCNT upper count limit
#define ENCODER_PCNT_LOW_LIMIT  -1000 // PCNT lower count limit
```

Encoder ID enumeration:

```
typedef enum _encoder_id {
    ENCODER_ID_M1 = 1,    // M1 encoder
    ENCODER_ID_M2 = 2,    // M2 encoder
} encoder_id;
```

PCNT initialization — quadrature decoding principle:

Quadrature decoding of the encoder requires two signals (phase A and phase B), which differ in phase by 90 degrees. By judging the phase relationship between A/B, the PCNT peripheral can simultaneously detect the rotation direction and the pulse count:

- Phase A leads phase B → forward rotation → the count increases
- Phase B leads phase A → reverse rotation → the count decreases

Encoder initialization (`Encoder_M1_Init` / `Encoder_M2_Init`):

Taking M1 as an example, the PCNT configuration flow is as follows:

1. **Create the PCNT unit:**

```
pcnt_unit_config_t unit_config = {
    .high_limit = ENCODER_PCNT_HIGH_LIMIT,    // Upper limit 1000
    .low_limit = ENCODER_PCNT_LOW_LIMIT,      // Lower limit -1000
    .flags.accum_count = true,                // Enable accumulate count mode
};
pcnt_new_unit(&unit_config, &pcnt_unit);
```

2. Configure the glitch filter (filters out high-frequency interference):

```
pcnt_glitch_filter_config_t filter_config = {
    .max_glitch_ns = 1000, // Filter out glitches narrower than 1000ns
};
pcnt_unit_set_glitch_filter(pcnt_unit, &filter_config);
```

3. Configure channel A (phase A as the edge signal, phase B as the level signal):

```
pcnt_chan_config_t chan_a_config = {
    .edge_gpio_num = ENCODER_GPIO_H1A,    // Edge input: GPIO5
    .level_gpio_num = ENCODER_GPIO_H1B,    // Level input: GPIO4
};
```

4. Configure channel B (phase B as the edge signal, phase A as the level signal):

```
pcnt_chan_config_t chan_b_config = {
    .edge_gpio_num = ENCODER_GPIO_H1B,    // Edge input: GPIO4
    .level_gpio_num = ENCODER_GPIO_H1A,    // Level input: GPIO5
};
```

5. Set the edge and level actions (the core of quadrature decoding):

The edge and level action configuration for channel A:

```
pcnt_channel_set_edge_action(chan_a, PCNT_CHANNEL_EDGE_ACTION_DECREASE,
PCNT_CHANNEL_EDGE_ACTION_INCREASE);
pcnt_channel_set_level_action(chan_a, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
```

- When the level (phase B) is low: the rising edge of phase A decrements the count, and the falling edge increments it
- When the level (phase B) is high: the increment/decrement direction is inverted
(`PCNT_CHANNEL_LEVEL_ACTION_INVERSE`)

The edge and level action configuration for channel B:

```
pcnt_channel_set_edge_action(chan_b, PCNT_CHANNEL_EDGE_ACTION_INCREASE,
PCNT_CHANNEL_EDGE_ACTION_DECREASE);
pcnt_channel_set_level_action(chan_b, PCNT_CHANNEL_LEVEL_ACTION_KEEP,
PCNT_CHANNEL_LEVEL_ACTION_INVERSE);
```

6. Add watch points so that the count automatically resets and accumulates when it exceeds the limits:

```
pcnt_unit_add_watch_point(pcnt_unit, ENCODER_PCNT_HIGH_LIMIT);
pcnt_unit_add_watch_point(pcnt_unit, ENCODER_PCNT_LOW_LIMIT);
```

7. Enable and start the PCNT:

```
pcnt_unit_enable(pcnt_unit);
pcnt_unit_clear_count(pcnt_unit);    // clear the count value
pcnt_unit_start(pcnt_unit);
```

Reading the encoder count (`Encoder_Get_Count_M1` / `Encoder_Get_Count_M2`):

```
int Encoder_Get_Count_M1(void)
{
    static int count_m1 = 0;
    pcnt_unit_get_count(encoder_unit_m1, &count_m1);
    return count_m1;
}
```

- Get the current count value of the PCNT unit via `pcnt_unit_get_count`
- The count increases during forward rotation and decreases during reverse rotation
- Since `accum_count` mode is enabled, the count automatically resets and accumulates when crossing the upper/lower limits

API functions:

Function	Description
<code>void Encoder_Init(void)</code>	Initialize the PCNT modules of the two motor encoders
<code>int Encoder_Get_Count_M1(void)</code>	Get the accumulated pulse count of the M1 motor encoder
<code>int Encoder_Get_Count_M2(void)</code>	Get the accumulated pulse count of the M2 motor encoder
<code>int Encoder_Get_Count(uint8_t encoder_id)</code>	Get the corresponding pulse count according to the encoder ID

Common Problems

Problem	Cause	Solution
The encoder value is always 0	Poor encoder wiring or the motors are not rotating	Check whether the encoder interface is plugged in firmly; rotate the wheels manually to observe the value changes

Problem	Cause	Solution
Abnormal jumps in the encoder value	Glitch interference or poor wiring contact	Check the encoder wiring, or try increasing the <code>max_glitch_ns</code> value of the glitch filter
The signs/directions of M1 and M2 encoders are opposite	This is normal (mirrored mounting)	The two motors are mounted in opposite directions on the chassis; apply the corresponding positive/negative mapping when reading the encoder signs
The encoder value stops changing after reaching ± 1000	The accumulate count reaches the PCNT upper/lower limits	Watch points have been added and <code>accum_count</code> mode enabled in the program, so the count should continue accumulating automatically after reaching the limits. If the problem persists, increase <code>ENCODER_PCNT_HIGH_LIMIT</code> and <code>ENCODER_PCNT_LOW_LIMIT</code>